

Chapter 32

External Memory Encryption and Decryption (XTS_AES)

32.1 Overview

The ESP32-P4 integrates an External Memory Encryption and Decryption module that complies with the XTS-AES standard algorithm specified in [IEEE Std 1619-2007](#), providing security for users' application code and data stored in the external memory (flash and RAM). Users can store proprietary firmware and sensitive data (e.g., credentials for gaining access to a private network) in the external flash, or store general data in the external RAM.

32.2 Features

- General XTS-AES algorithm, compliant with IEEE Std 1619-2007
- Software-based manual encryption
- High-speed auto encryption and decryption without software's participation
- Encryption and decryption functions jointly enabled/disabled by registers configuration, eFuse parameters, and boot mode
- Configurable Anti-DPA

32.3 Module Structure

The External Memory Encryption and Decryption module consists of three blocks, namely the Manual Encryption block, Auto Encryption block, and Auto Decryption block. The module architecture is shown in Figure [32.3-1](#).

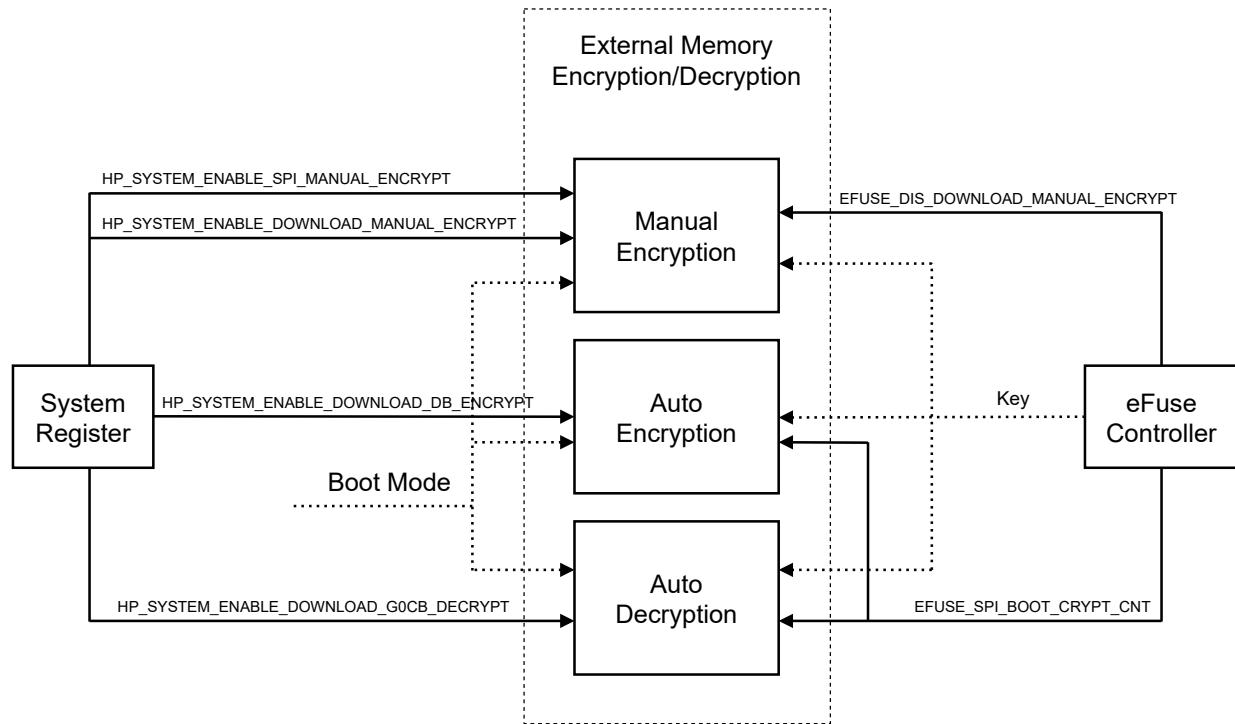


Figure 32.3-1. Architecture of the External Memory Encryption and Decryption

The Manual Encryption block can encrypt instructions and data, which will then be written to the external flash as ciphertext via SPI1.

When the CPU writes data to the external RAM through cache, the Auto Encryption block will automatically encrypt the data first, then the data will be written to the external RAM as ciphertext.

When the CPU reads from the external flash or external RAM through cache, the Auto Decryption block will automatically decrypt the ciphertext to retrieve instructions and data.

In the System Registers peripheral (see [20 System Registers \(SYSREG\)](#)), the following four bits in register `HP_SYSTEM_CRYPT_CTRL_REG` are relevant to the external memory encryption and decryption:

- `HP_SYSTEM_ENABLE_DOWNLOAD_MANUAL_ENCRYPT`
- `HP_SYSTEM_ENABLE_DOWNLOAD_GOCB_DECRYPT`
- `HP_SYSTEM_ENABLE_DOWNLOAD_DB_ENCRYPT`
- `HP_SYSTEM_ENABLE_SPI_MANUAL_ENCRYPT`

The XTS_AES module also fetches two parameters from the peripheral eFuse Controller, which are: `EFUSE_DIS_DOWNLOAD_MANUAL_ENCRYPT` and `EFUSE_SPI_BOOT_CRYPT_CNT`. For detailed information, please see Chapter 8 [eFuse Controller \(EFUSE\)](#).

32.4 Functional Description

32.4.1 XTS Algorithm

The manual encryption and auto encryption/decryption all use the same algorithm. During implementation, the XTS algorithm is characterized by a "data unit" of 1024 bits, defined in the Section *XTS-AES encryption procedure* of

[XTS-AES Tweakable Block Cipher](#) Standard. For more information about the XTS-AES algorithm, please refer to [IEEE Std 1619-2007](#).

32.4.2 Key

The Manual Encryption block, Auto Encryption block, and Auto Decryption block share the same *Key* when implementing the XTS algorithm. The *Key* is provided by the eFuse hardware and cannot be accessed by users.

The *Key* can be either 256-bit or 512-bit long. The value and length of the *Key* are determined by the content in the eFuse blocks from Block4 ~ Block9 and corresponding eFuse parameters. For easier description, we define:

- Block_A: the block whose key purpose is EFUSE_KEY_PURPOSE_XTS_AES_256_KEY_1 (please refer to Table 8.3-2 *Secure Key Purpose Values*). If Block_A exists, a 256-bit *Key_A* is stored in it.
- Block_B: the block whose key purpose is EFUSE_KEY_PURPOSE_XTS_AES_256_KEY_2 (please refer to Table 8.3-2 *Secure Key Purpose Values*). If Block_B exists, a 256-bit *Key_B* is stored in it.
- Block_C: the block whose key purpose is EFUSE_KEY_PURPOSE_XTS_AES_128_KEY (please refer to Table 8.3-2 *Secure Key Purpose Values*). If Block_C exists, a 256-bit *Key_C* is stored in it.

There are five possibilities of how the *Key* is generated depending on whether Block_A, Block_B, and Block_C exist or not, as shown in Table 32.4-1. In each case, the *Key* can be uniquely determined by Block_A, Block_B or Block_C.

Table 32.4-1. Key generated based on *Key_A*, *Key_B*, and *Key_C*

Block _A	Block _B	Block _C	<i>Key</i>	<i>Key</i> Length (bit)
Yes	Yes	Don't care	<i>Key_A</i> <i>Key_B</i>	512
Yes	No	Don't care	<i>Key_A</i> 0 ²⁵⁶	512
No	Yes	Don't care	0 ²⁵⁶ <i>Key_B</i>	512
No	No	Yes	<i>Key_C</i>	256
No	No	No	0 ²⁵⁶	256

Notes:

- "YES" indicates that the block exists
- "NO" indicates that the block does not exist
- "0²⁵⁶" indicates a bit string that consists of 256-bit zeros
- "||" is a bonding operator for joining one-bit string to another

For more information on key purposes, please refer to Table 8.3-2 *Secure Key Purpose Values* in Chapter 8 *eFuse Controller (EFUSE)*.

32.4.3 Target Memory Space

The target memory space refers to a continuous address space in the external memory where the first encrypted ciphertext is stored. The target memory space can be uniquely determined by three relevant parameters: type, size, and base address, whose definitions are listed below.

- Type: the *type* of the target memory space, either external flash or external RAM. Value 0 indicates external flash, while 1 indicates external RAM.
- Size: the *size* of the target memory space, indicating the number of bytes encrypted in one encryption operation, which supports 16, 32, or 64 bytes.
- Base address: the *base_addr* of the target memory space. It is a 30-bit physical address, with a range of 0x0000_0000 ~ 0x3FFF_FFFF. It should be aligned to *size*, i.e., $base_addr \% size == 0$.

For example, if there are 16 bytes of instruction data that need to be encrypted and written to address 0x130 ~ 0x13F in the external flash, then the target space is 0x130 ~ 0x13F, type is 0 (external flash), size is 16 (bytes), and the base address is 0x130.

The encryption of any length (must be multiples of 16 bytes) of plaintext instruction/data can be completed separately in multiple operations, and each operation has individual target memory space and relevant parameters.

For Auto Encryption/Decryption blocks, these parameters are automatically defined by hardware. For the Manual Encryption block, these parameters should be configured manually by users.

Note:

The “tweak” defined in Chapter 5.1 *Data units and tweaks* of [IEEE Std 1619-2007](#) is a 128-bit non-negative integer (*tweak*), which can be generated according to $tweak = type * 2^{30} + (base_addr \& 0x3FFFFFF80)$. The lowest 7 bits and the highest 97 bits in *tweak* are always zero.

32.4.4 Data Writing

For Auto Encryption/Decryption blocks, data writing is automatically applied in hardware. For Manual Encryption blocks, data writing should be applied by users. The Manual Encryption block has a register block which consists of 16 registers, i.e., `XTS_AES_PLAIN_n_REG` (n : 0 ~ 15), that are dedicated to data writing and can store up to 512 bits of plaintext at a time.

Actually, the Manual Encryption block does not care where the plaintext comes from, but only where the ciphertext will be stored. Because of the strict correspondence between plaintext and ciphertext, in order to better describe how the plaintext is stored in the register block, we assume that the plaintext is stored in the target memory space in the first place and replaced by ciphertext after encryption. Therefore, the following description in this section no longer has the concept of “plaintext”, but uses “target memory space” instead.

How mapping between target memory space and registers works:

Assume a word in the target memory space is stored in *address*, define $offset = address \% 64$, $n = offset / 4$, then the word will be stored in register `XTS_AES_PLAIN_n_REG`.

For example, when the *size* is 64, all registers in the register block will be used. The mapping between *offset* and registers now is shown in Table 32.4-2.

Table 32.4-2. Mapping Between Offsets and Registers

<i>offset</i>	Register	<i>offset</i>	Register
0x00	XTS_AES_PLAIN_0_REG	0x20	XTS_AES_PLAIN_8_REG
0x04	XTS_AES_PLAIN_1_REG	0x24	XTS_AES_PLAIN_9_REG
0x08	XTS_AES_PLAIN_2_REG	0x28	XTS_AES_PLAIN_10_REG
0x0C	XTS_AES_PLAIN_3_REG	0x2C	XTS_AES_PLAIN_11_REG
0x10	XTS_AES_PLAIN_4_REG	0x30	XTS_AES_PLAIN_12_REG
0x14	XTS_AES_PLAIN_5_REG	0x34	XTS_AES_PLAIN_13_REG
0x18	XTS_AES_PLAIN_6_REG	0x38	XTS_AES_PLAIN_14_REG
0x1C	XTS_AES_PLAIN_7_REG	0x3C	XTS_AES_PLAIN_15_REG

32.4.5 Manual Encryption Block

The Manual Encryption block is a peripheral module. It is equipped with registers and can be accessed by the CPU directly. Registers embedded in this block, the System Registers peripheral, eFuse parameters, and boot mode jointly configure and use this module. Please note that the Manual Encryption block can only encrypt for storage in the external flash.

The Manual Encryption block is operational only under certain conditions:

- In SPI Boot mode:

If bit [HP_SYSTEM_ENABLE_SPI_MANUAL_ENCRYPT](#) in register

[HP_SYSTEM_CRYPT_CTRL_REG](#) is 1, the Manual Encryption block can be enabled. Otherwise, it is not operational.

- In Joint Download Boot mode:

If bit [HP_SYSTEM_ENABLE_DOWNLOAD_MANUAL_ENCRYPT](#) in register

[HP_SYSTEM_CRYPT_CTRL_REG](#) is 1 and the eFuse parameter

[EFUSE_DIS_DOWNLOAD_MANUAL_ENCRYPT](#) is 0, the Manual Encryption block can be enabled.

Otherwise, it is not operational.

Note:

Even though the CPU can skip cache and get the encrypted instruction/data directly by reading the external memory, users can by no means access *Key*.

32.4.6 Auto Encryption Block

The Auto Encryption block is not a conventional peripheral, so it does not have any registers and cannot be accessed by the CPU directly. The System Registers peripheral, eFuse parameters, and boot mode jointly configure and use this block.

The Auto Encryption block is operational only under certain conditions:

- In SPI Boot mode:

when [EFUSE_SPI_BOOT_CRYPT_CNT](#) (3 bits) is set to 1, 2, 4 or 7 (i.e., there is an odd number of 1s in its binary representation), then the Auto Encryption block can be enabled. Otherwise, it is not operational.

- In Joint Download Boot mode:

when bit [HP_SYSTEM_ENABLE_DOWNLOAD_DB_ENCRYPT](#) in register [HP_SYSTEM_CRYPT_CTRL_REG](#) is 1, the Auto Encryption block can be enabled. Otherwise, it is not operational.

Note:

- When the Auto Encryption block is enabled, it will automatically encrypt data if the CPU writes data to the external RAM, and then the encrypted ciphertext will be written to the external RAM. The entire encryption process does not need software participation and is transparent to the cache. Users can by no means obtain the encryption *Key* during the process.
- When the Auto Encryption block is disabled, it will ignore the CPU's access request to the cache and not process the data. Therefore, the data will be written to the external RAM as plaintext directly.

32.4.7 Auto Decryption Block

The Auto Decryption block is not a conventional peripheral, so it does not have any registers and cannot be accessed by the CPU directly. The System Registers peripheral, eFuse parameters, and boot mode jointly configure and use this block.

The Auto Decryption block is operational only under certain conditions:

- In SPI Boot mode:

when [EFUSE_SPI_BOOT_CRYPT_CNT](#) (3 bits in total) is set to 1, 2, 4 or 7 (i.e., there is an odd number of 1s in its binary representation), then the Auto Decryption block can be enabled. Otherwise, it is not operational.

- In Joint Download Boot mode:

when bit [HP_SYSTEM_ENABLE_DOWNLOAD_GOCB_DECRYPT](#) in register [HP_SYSTEM_CRYPT_CTRL_REG](#) is 1, the Auto Decryption block can be enabled. Otherwise, it is not operational.

Note:

- When the Auto Decryption block is enabled, it will automatically decrypt the ciphertext if the CPU reads instructions/data from the external memory via cache to retrieve the instructions/data. The entire decryption process does not need software participation and is transparent to the cache. The software can by no means obtain the decryption *Key* during the process.
- When the Auto Decryption block is disabled, it does not have any effect on the contents stored in the external memory, no matter if they are encrypted or not. Therefore, what the CPU reads via cache is the original information stored in the external memory.

32.5 Software Process

When the Manual Encryption block operates, software needs to be involved in the process. The steps are as follows:

1. Configure XTS_AES:

- Set register [XTS_AES_DESTINATION_REG](#) to *type* = 0.
- Set register [XTS_AES_PHYSICAL_ADDRESS_REG](#) to *base_addr*.
- Set register [XTS_AES_LINESIZE_REG](#) to $\frac{size}{32}$.

For definitions of *base_addr* and *size*, please refer to Section [32.4.3](#).

2. Write plaintext instructions/data to the registers block XTS_AES_PLAIN_*n*_REG (*n*: 0-15). For detailed information, please refer to Section [32.4.4](#).

Please write data to registers according to your actual needs, and the unused ones could be set to arbitrary values.

3. Wait for Manual Encryption block to be idle. Poll register [XTS_AES_STATE_REG](#) until it reads 0 which indicates the Manual Encryption block is idle.4. Trigger manual encryption by writing 1 to register [XTS_AES_TRIGGER_REG](#).5. Wait for the encryption process completion. Poll register [XTS_AES_STATE_REG](#) until it reads 2.

Step 1 to 5 are the steps of encrypting plaintext instructions/data with the Manual Encryption block using the Key.

6. Write 1 to register [XTS_AES_RELEASE_REG](#) to grant SPI1 the access to the encrypted ciphertext. After this, the value of register [XTS_AES_STATE_REG](#) will become 3.7. Call SPI1 to write the ciphertext in the external flash (see Section [API Reference - Flash Encrypt](#) in [ESP-IDF Programming Guide](#)).8. Write 1 to register [XTS_AES_DESTROY_REG](#) to destroy the ciphertext. After this, the value of register [XTS_AES_STATE_REG](#) will become 0.

Repeat the above steps according to the amount of plaintext instructions/data that need to be encrypted.

32.6 Anti-DPA

DPA (Differential Power Analysis) is a side-channel attack method in cryptography, through which an attacker can statistically analyze data collected from multiple encryption operations to calculate intermediate values in the encryption computation. ESP32-P4 XTS_AES supports Anti-DPA to defend against external DPA attacks.

The XTS-AES algorithm can be divided into two steps, according to [IEEE Std 1619-2007](#):

- Step 1: Calculating T value. In this section, we define this step as "calculating T".
- Step 2: Calculating Cipher/Plain text. In this section, we define this step as "calculating D".

Different security levels can be configured through registers:

- First we define the below parameters for a better description:
 - *select_reg* = [XTS_AES_CRYPT_DPA_SELECT_REGISTER](#)
 - *reg_d_dpa_en* = [XTS_AES_CRYPT_CALC_D_DPA_EN](#)
 - *efuse_dpa_en* = [EFUSE_CRYPT_DPA_ENABLE](#)

- $reg_anti_dpa_level = XTS_AES_CRYPT_SECURITY_LEVEL$
- $efuse_anti_dpa_level = 3$

- Configure the security level of Anti-DPA for the XTS_AES module:

$$Anti_DPA_level = select_reg ? (reg_anti_dpa_level) : (efuse_dpa_en * efuse_anti_dpa_level)$$

When $Anti_DPA_level$ equals 0, Anti-DPA is disabled. The higher the value of $Anti_DPA_level$ is, the stronger the Anti-DPA ability is.

- Configure whether to enable Anti-DPA when the XTS-AES algorithm is calculating D:

$$Anti_DPA_enabled_in_calc_D = select_reg ? reg_d_dpa_en : efuse_dpa_en$$

If $Anti_DPA_level$ is not 0, when $Anti_DPA_enabled_in_calc_D$ equals to 1, Anti-DPA is enabled when XTS-AES algorithm is calculating D.

If $Anti_DPA_level$ is not 0, Anti-DPA is always enabled when the XTS-AES algorithm is calculating T.

Note:

- Even if $efuse_dpa_en$ is set to 1, you can still disable anti-DPA by configuring $select_reg = 1$ and $reg_anti_dpa_level = 0$.
- Configuring whether or not to enable Anti-DPA will have an impact on the external storage access bandwidth:
 - When Anti-DPA is enabled during the calculation of D, the read and write bandwidth will be reduced by more than 50% when the Anti-DPA level ≥ 4 .
 - When Anti-DPA is disabled during the calculation of D, the read and write bandwidth will be reduced by more than 50% when the Anti-DPA level ≥ 6 .

32.7 Register Summary

The addresses in this section are relative to the External Memory Encryption and Decryption base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

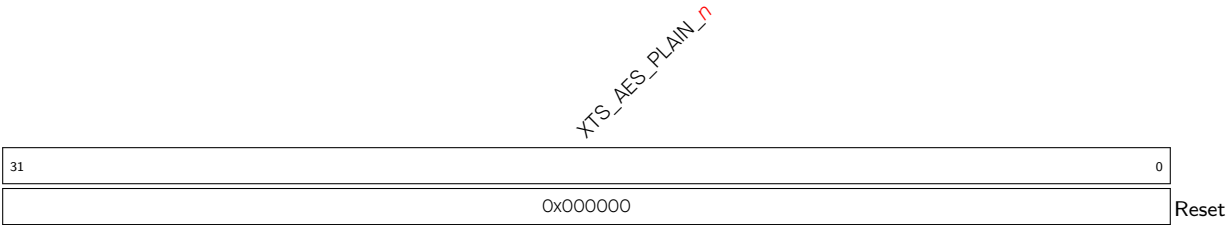
Name	Description	Address	Access
Plaintext Register Heap			
XTS_AES_PLAIN_0_REG	Plaintext register 0	0x0300	R/W
XTS_AES_PLAIN_1_REG	Plaintext register 1	0x0304	R/W
XTS_AES_PLAIN_2_REG	Plaintext register 2	0x0308	R/W
XTS_AES_PLAIN_3_REG	Plaintext register 3	0x030C	R/W
XTS_AES_PLAIN_4_REG	Plaintext register 4	0x0310	R/W
XTS_AES_PLAIN_5_REG	Plaintext register 5	0x0314	R/W
XTS_AES_PLAIN_6_REG	Plaintext register 6	0x0318	R/W
XTS_AES_PLAIN_7_REG	Plaintext register 7	0x031C	R/W
XTS_AES_PLAIN_8_REG	Plaintext register 8	0x0320	R/W
XTS_AES_PLAIN_9_REG	Plaintext register 9	0x0324	R/W
XTS_AES_PLAIN_10_REG	Plaintext register 10	0x0328	R/W
XTS_AES_PLAIN_11_REG	Plaintext register 11	0x032C	R/W
XTS_AES_PLAIN_12_REG	Plaintext register 12	0x0330	R/W
XTS_AES_PLAIN_13_REG	Plaintext register 13	0x0334	R/W
XTS_AES_PLAIN_14_REG	Plaintext register 14	0x0338	R/W
XTS_AES_PLAIN_15_REG	Plaintext register 15	0x033C	R/W
Configuration Registers			
XTS_AES_LINESIZE_REG	Configures the size of target memory space	0x0340	R/W
XTS_AES_DESTINATION_REG	Configures the type of the external memory	0x0344	R/W
XTS_AES_PHYSICAL_ADDRESS_REG	Physical address	0x0348	R/W
XTS_AES_DPA_CTRL_REG	Configures the Anti-DPA function	0x0388	R/W
Control/Status Registers			
XTS_AES_TRIGGER_REG	Activates AES algorithm	0x034C	WO
XTS_AES_RELEASE_REG	Release control	0x0350	WO
XTS_AES_DESTROY_REG	Destroy control	0x0354	WO
XTS_AES_STATE_REG	Status register	0x0358	RO
Version Register			
XTS_AES_DATE_REG	Version control register	0x035C	R/W

32.8 Registers

The addresses in this section are relative to the External Memory Encryption and Decryption base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

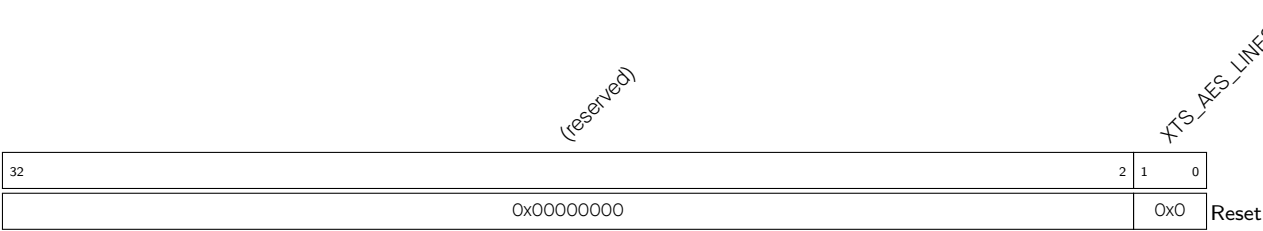
For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 32.1. XTS_AES_PLAIN_ *n* _REG (*n*: 0-15) (0x0300+4**n*)



XTS_AES_PLAIN_ *n* Configures the *n*th 32-bit piece of plain text. (R/W)

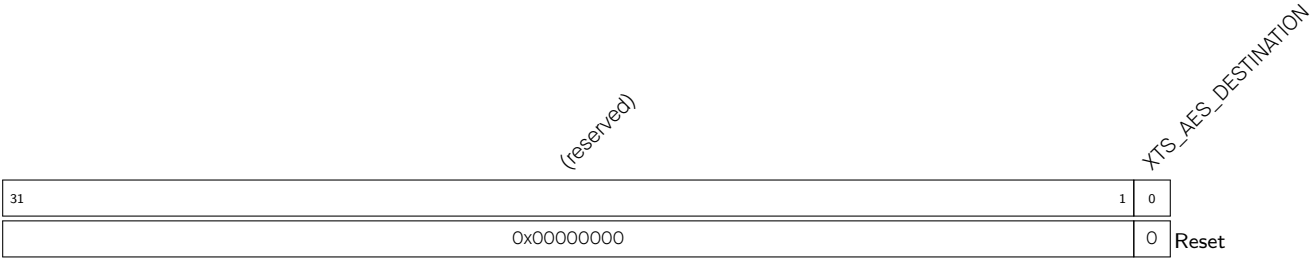
Register 32.2. XTS_AES_LINESIZE_REG (0x0340)



XTS_AES_LINESIZE Configures the data size of one encryption operation.

- 0: 16 bytes
 - 1: 32 bytes
 - 2: 64 bytes
 - 3: Invalid
- (R/W)

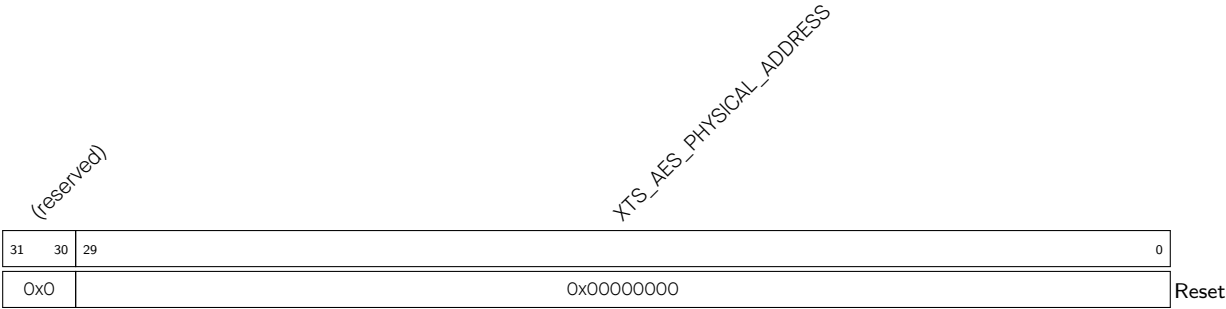
Register 32.3. XTS_AES_DESTINATION_REG (0x0344)



XTS_AES_DESTINATION Configures the type of external memory for Manual Encryption. Currently, it must be set to 0, as the Manual Encryption block only supports flash encryption. Set this bit to 1 may cause an error.

0: flash
1: external RAM
(R/W)

Register 32.4. XTS_AES_PHYSICAL_ADDRESS_REG (0x0348)



XTS_AES_PHYSICAL_ADDRESS Configures physical address. Note that its value should be within the range between 0x0000_0000 and 0x00FF_FFFF. (R/W)

Register 32.5. XTS_AES_DPA_CTRL_REG (0x0388)

(reserved)																XTS_AES_CRYPT_DPA_SELECT_REGISTER		XTS_AES_CRYPT_CALC_D_DPA_EN		XTS_AES_CRYPT_SECURITY_LEVEL	
31																5	4	3	2	0	
0x00000000																0x0	0x1	0x7		Reset	

XTS_AES_CRYPT_DPA_SELECT_REGISTER Configures whether the Anti-DPA function is controlled by eFuse or register.

0: The Anti-DPA function is configured by register.

1: The Anti-DPA function is configured by eFuse.

(R/W)

XTS_AES_CRYPT_CALC_D_DPA_EN Configures whether to enable Anti-DPA in the XTS_AES algorithm.

0: Enable Anti-DPA only when calculating T

1: Enable Anti-DPA both when calculating T and D

Note that this field is only effective when [XTS_AES_CRYPT_SECURITY_LEVEL](#) is not 0.

(R/W)

XTS_AES_CRYPT_SECURITY_LEVEL Configures the security level of external memory encryption and decryption.

0: Disable the Anti-DPA function

1-7: The bigger the number is, the more secure the encryption and decryption are

(R/W)

Register 32.6. XTS_AES_TRIGGER_REG (0x034C)

(reserved)																XTS_AES_TRIGGER	
31															1	0	
0x00000000																x	Reset

XTS_AES_TRIGGER Configures whether or not to enable manual encryption.

0: Disable manual encryption

1: Enable manual encryption

(WO)

Register 32.7. XTS_AES_RELEASE_REG (0x0350)

(reserved)																XTS_AES_RELEASE	
31															1	0	Reset
0x00000000																x	

XTS_AES_RELEASE Configures whether to grant SPI1 access to the encrypted result.

0: No effect

1: Grant SPI1 access

(WO)

Register 32.8. XTS_AES_DESTROY_REG (0x0354)

(reserved)																XTS_AES_DESTROY	
31																1	0
0x00000000																x	Reset

XTS_AES_DESTROY Configures whether to destroy the encrypted result.

0: No effect

1: Destroy encrypted result

(WO)

Register 32.9. XTS_AES_STATE_REG (0x0358)

(reserved)																XTS_AES_STATE		
31															2	1	0	
0x00000000																0x0		Reset

XTS_AES_STATE Represents the status of the Manual Encryption block. 0 (XTS_AES_IDLE): Idle

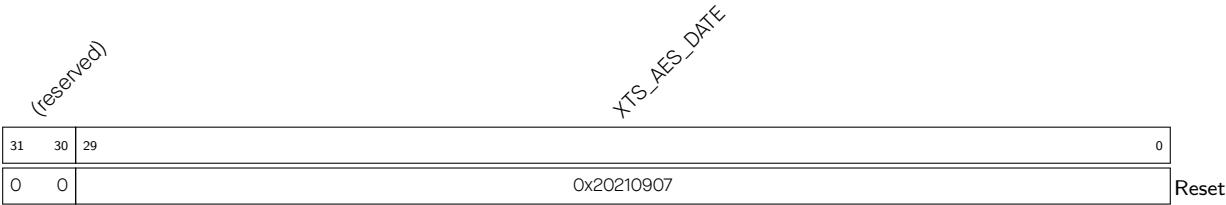
1 (XTS_AES_BUSY): Busy with encryption

2 (XTS_AES_DONE): Encryption completed, but the encrypted result is not accessible to SPI

3 (XTS_AES_RELEASE): Encrypted result is accessible to SPI

(RO)

Register 32.10. XTS_AES_DATE_REG (0x035C)



XTS_AES_DATE Version control register. (R/W)